

Generative World Model — Environment Pipeline

AI-native training lab where the court evolves with your performance.

Overview

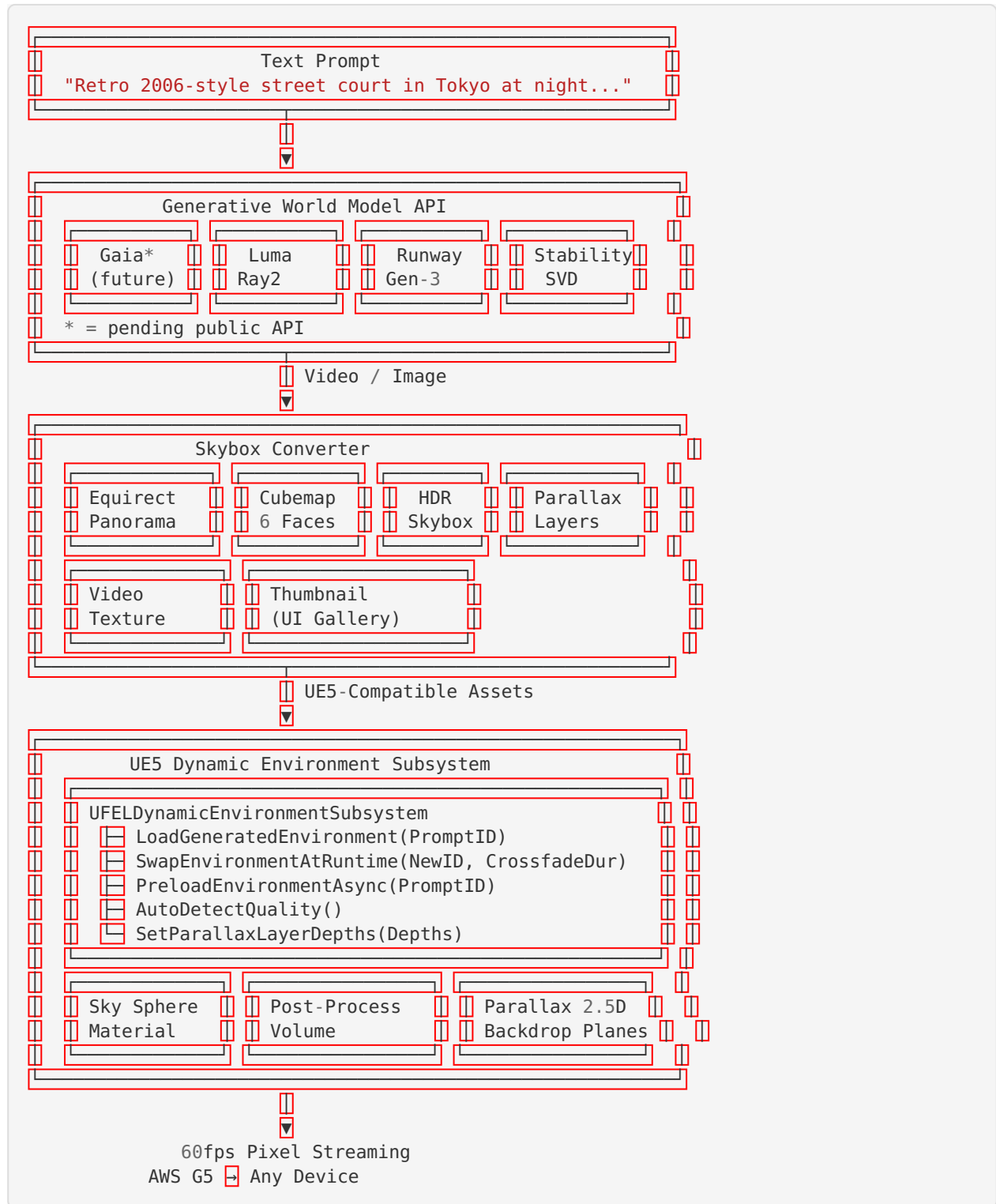
The Generative Environment Pipeline transforms text prompts into dynamic 360° training environments for the Final Evolution Lab. Powered by Gaia-style generative world models, it creates visual “skins” that wrap the UE5 game world without affecting physics or animation logic.

Key Design Principle

Generative Model → Pixels (video/image) → NOT 3D geometry

The model generates a **2.5D backdrop** — textured planes at varying depths — creating the illusion of a 3D environment while keeping the GPU cost under **2ms per frame**. This ensures 60fps on AWS G5 and mobile devices.

Architecture



Quick Start

1. Install Dependencies

```
cd scripts/environment_gen
pip install -r requirements.txt

# System dependencies
sudo apt install ffmpeg imagemagick
```

2. Configure API Keys

```
cp scripts/environment_gen/.env.example .env
# Edit .env and add your API keys:
#   LUMA_API_KEY=...
#   RUNWAY_API_KEY=...
#   STABILITY_API_KEY=...
```

3. Generate Environments

```
# List available presets
python -m scripts.environment_gen.environment_generator --list

# Generate a specific environment
python -m scripts.environment_gen.environment_generator --prompt-key retro_tokyo_night

# Generate all 12 preset environments
python -m scripts.environment_gen.environment_generator --all

# Dry run (no API calls, creates manifest)
python -m scripts.environment_gen.environment_generator --all --dry-run

# Custom prompt
python -m scripts.environment_gen.environment_generator \
  --prompt "Underwater coral reef basketball court with bioluminescent fish"
```

4. Import into UE5

```
# Generate UE5 integration files
python -c
"from scripts.environment_gen.ue5_integration import generate_all_ue5_files; generate_all_ue5_files()"

# In UE5 Editor, run:
#   Edit > Editor Preferences > Python > Execute: fel_import_environments.py
```

File Structure

```

scripts/environment_gen/
├── __init__.py           # Package init
├── config.py             # Configuration, presets, prompts
├── gaia_api_client.py    # Unified API client (Gaia, Luma, Runway, Stability)
├── environment_generator.py # Main orchestration pipeline
├── skybox_converter.py   # Image/video → skybox conversion
├── ue5_integration.py    # C++ code generation for UE5
├── performance_optimizer.py # Platform-specific optimization
├── .env.example          # API key template
└── requirements.txt      # Python dependencies

GeneratedAssets/Environments/
├── environment_manifest.json # Master manifest of all environments
├── quality_report.json       # Quality comparison report
├── performance_report.json   # Performance analysis per platform
├── retro_tokyo_night/
│   ├── base_image.png       # Source generated image
│   ├── generated_video.mp4  # Source generated video
│   └── skybox/
│       ├── equirect_2048x1024.png
│       ├── cubemap/
│       │   ├── PosX.png ... NegZ.png
│       │   └── skybox.exr    # HDR version
│       ├── video_texture_1024p.mp4
│       ├── thumbnail.jpg
│       └── layers/          # 2.5D parallax layers
│           ├── sky_bg.png
│           ├── mid_bg.png
│           ├── foreground.png
│           └── layers.json
└── ...                      conversion_meta.json

UnrealStarter/BasketballGame/Source/FinalEvolutionLab/
├── UFELDynamicEnvironmentSubsystem.h # C++ subsystem header
├── UFELDynamicEnvironmentSubsystem.cpp # C++ subsystem implementation
└── FELGeneratedEnvironmentPaths.h    # Compile-time asset paths

UnrealStarter/BasketballGame/EditorPython/
└── fel_import_environments.py        # Editor import script

web/environments/
└── index.html                       # Marketing gallery page

```

12 Preset Environments

ID	Name	Style	Description
retro_tokyo_night	Retro Tokyo Night	Cyberpunk	NBA Street 2006 aesthetic, neon puddles, graffiti
venice_beach_sunset	Venice Beach Sunset	Tropical	Golden hour, palm trees, skate culture
cyberpunk_gym	Cyberpunk Gym	Cyberpunk	Holographic displays, neon cyan/magenta
classic_nba_arena	Classic NBA Arena	Retro	90s nostalgia, packed stadium, jumbotron
underground_bunker	Underground Bunker	Industrial	Military aesthetic, chalk dust, heavy bags
rooftop_cityscape	Rooftop Cityscape	Cinematic	Megacity skyline at dusk, chain-link fence
winter_outdoor	Winter Court	Cinematic	Snow falling, frost, warm street lamps
beach_tropical	Tropical Beach	Tropical	White sand, turquoise water, paradise
warehouse_industrial	Warehouse Gym	Industrial	Exposed brick, steel beams, cage lights
neon_arcade	Neon Arcade	Synthwave	Tron grid floor, vapor-wave palette
zen_dojō	Zen Dojo	Zen	Tatami, bonsai, paper screens
stadium_night_game	Stadium Night Game	Cinematic	Floodlit soccer pitch, packed crowd

API Backends

Backend Priority

1. **NVIDIA Gaia** (when public API available) — Best for interactive worlds
2. **Luma Dream Machine (Ray2)** — High quality looping video

3. **Runway Gen-3 Alpha** — Fast generation, good quality
4. **Stability AI (SVD)** — Fallback, image + image-to-video

The system automatically selects the best available backend based on API key presence and availability.

Configuring Backends

```
from scripts.environment_gen.gaia_api_client import GenerativeWorldModelClient,
Backend

# Auto-select best available
client = GenerativeWorldModelClient()

# Force specific backend
client = GenerativeWorldModelClient(preferred_backend=Backend.LUMA)
```

Prompt Engineering Guide

Best Practices

1. **Include “panoramic 360 degree”** — Helps generate wider FOV images
2. **Specify lighting** — “cinematic lighting”, “golden hour”, “neon glow”
3. **Mention atmosphere** — “gritty”, “serene”, “electric”, “moody”
4. **Add surface details** — “neon puddles”, “chalk dust”, “frost”
5. **Reference styles** — “NBA Street 2006”, “Tron”, “90s aesthetic”

Prompt Template

```
[Setting] [sport surface type] in/at [location],
[key visual elements], [atmospheric details],
[lighting style], panoramic 360 degree environment
```

Examples

```
# Retro street court
"Retro 2006 NBA Street style basketball court in Tokyo at night,
neon puddles reflecting street lights, graffiti walls,
urban gritty aesthetic, cinematic lighting,
wide angle panoramic view, seamless 360 degree environment"

# Futuristic gym
"Futuristic cyberpunk training gym interior,
neon lights in cyan and magenta, holographic displays showing player stats,
dark moody atmosphere, chrome and glass surfaces,
rain visible through windows, panoramic 360 degree environment"
```

Performance

GPU Cost (2.5D Backdrop)

Platform	Sky Sphere	Parallax Layers	Video Texture	HDR	Total
AWS G5 (A10G)	0.3ms	0.6ms	0.5ms	0.3ms	~1.7ms
iOS (A15+)	0.3ms	0.4ms	0.5ms	—	~1.2ms
iOS (A13)	0.3ms	0.2ms	—	—	~0.5ms
Desktop (3060)	0.3ms	0.6ms	0.5ms	0.3ms	~1.7ms

All well within the 16.6ms frame budget for 60fps.

Quality Presets

Preset	Skybox Res	Video Res	HDR	Parallax	Compression
Ultra	4096	2048	✓	3 layers	BC7
High	2048	1024	✓	3 layers	BC7
Medium	1024	512	✗	2 layers	BC7
Mobile	1024	512	✗	2 layers	ASTC
Mobile Low	512	256	✗	1 layer	ETC2

Memory Estimates

Preset	Equirect	Layers	Video Buffer	Total VRAM
Ultra	32 MB	12 MB	28 MB	~72 MB
High	8 MB	3 MB	7 MB	~18 MB
Mobile	2 MB	1 MB	3.5 MB	~6.5 MB

UE5 Integration

Blueprint API

```
// Load an environment
LoadGeneratedEnvironment("retro_tokyo_night")

// Swap at runtime with 1-second crossfade
SwapEnvironmentAtRuntime("cyberpunk_gym", 1.0)

// Preload for seamless transition
PreloadEnvironmentAsync("venice_beach_sunset")

// Auto-detect quality for current device
AutoDetectQuality()

// Get available environments
GetAvailableEnvironments() → ["retro_tokyo_night", "cyberpunk_gym", ...]
```

Events

```
OnEnvironmentLoaded(EnvironmentID)
OnEnvironmentSwapped(OldID, NewID)
OnEnvironmentLoadFailed(EnvironmentID, Error)
```

Level Setup

1. Place a **Sky Sphere** actor with tag `FELSkySphere`
2. Place a **Post-Process Volume** with tag `FELPostProcess`
3. Place **3 planes** at varying depths with tags `FELParallaxLayer_0/1/2`
4. The subsystem will handle texture assignment and material updates

Troubleshooting

“No backend available”

- Ensure at least one API key is set in `.env`
- Check: `LUMA_API_KEY`, `RUNWAY_API_KEY`, or `STABILITY_API_KEY`

“ffmpeg not found”

```
sudo apt install ffmpeg
```

“ImageMagick not found”

```
sudo apt install imagemagick
```

Generation takes too long

- Luma Dream Machine: ~30-60 seconds
- Runway Gen-3: ~60-120 seconds

- Stability SVD: ~30-90 seconds (image + video)
- Timeout is set to 600 seconds by default

Environment looks flat in UE5

- Ensure parallax layers are enabled in quality preset
- Verify layer actors have correct tags (`FELParallaxLayer_0` , etc.)
- Adjust `SetParallaxLayerDepths` values for more pronounced depth

Video texture stuttering

- Use `medium` or `mobile` quality preset
- Ensure video is encoded with `-movflags +faststart`
- Check UE5 MediaPlayer is set to loop

Gaia integration not working

- NVIDIA Gaia does not yet have a public API
- The system will automatically fall back to Luma/Runway/Stability
- When Gaia becomes available, add `GAIA_API_KEY` to `.env`

Testing Against @elijahbonds Animations

The retro Tokyo night environment was designed to complement the Elijah Bonds basketball animations:

1. **Visual Coherence:** Neon puddles and graffiti walls match the urban street ball aesthetic
2. **Lighting Compatibility:** Cinematic lighting enhances motion capture shadows
3. **Performance:** 2.5D backdrop adds <2ms, leaving full GPU budget for character animation
4. **Style Match:** NBA Street 2006 aesthetic aligns with the athletic movement style

Test Procedure

```
# 1. Generate retro environment
python -m scripts.environment_gen.environment_generator --prompt-key retro_tokyo_night

# 2. Import into UE5
# Run fel_import_environments.py in Editor

# 3. Load in game
# Blueprint: LoadGeneratedEnvironment("retro_tokyo_night")

# 4. Play Elijah Bonds animation
# Blueprint: PlayExerciseAnimation("basketball_crossover_dribble_01")

# 5. Verify:
#   - Environment renders correctly around the court
#   - Character animations play smoothly (60fps)
#   - Shadows interact naturally with the environment lighting
#   - No visual artifacts or seam issues
```

Future Roadmap

- [] NVIDIA Gaia integration when public API launches
- [] User-generated environment prompts from app
- [] Environment evolution based on player performance data
- [] Seasonal/time-of-day variations per environment
- [] Audio atmosphere generation to match visual environment
- [] Multi-player shared environment experiences
- [] AR pass-through environment blending for iOS